

Projet de Base de Données

Gestion d'une Plateforme de Tournois E-sport

Halim Djerroud

2026

Informations générales	
Niveau :	LSI
Format :	Binôme
Date de rendu :	15/04/2026
Modalité :	Archive .zip déposée sur Moodle
Soutenance :	Orale de 15 minutes (démonstration + questions) à la dernière séance

Table des matières

1. Présentation du sujet	4
2. Description du domaine	4
2.1. Les jeux	4
2.2. Les joueurs	4
2.3. Les équipes	4
2.4. Les tournois	4
2.5. Les matchs	4
2.6. Les statistiques	5
3. Travail demandé	5
I. Modélisation	6
4. Modèle Conceptuel de Données (MCD)	6
4.1. Travail à réaliser	6
4.2. Contraintes obligatoires	6
4.3. Livrable attendu	6
5. Modèle Logique de Données (MLD)	6
5.1. Travail à réaliser	7
5.2. Livrable attendu	7
II. Base de données SQL	8
6. Script de création (DDL)	8
6.1. Travail à réaliser	8
6.2. Exemple de structure attendue	8
6.3. Jeu de données (DML)	8
7. Requêtes SQL	9
7.1. Requêtes de base (SELECT, WHERE, ORDER BY)	9
7.2. Requêtes avec jointures	9
7.3. Requêtes avec agrégats (GROUP BY, HAVING)	9
7.4. Requêtes avancées (sous-requêtes, EXISTS, NOT EXISTS)	9
III. Application Java	11
8. Architecture de l'application	11
8.1. Connexion JDBC	11
8.2. Organisation du code	11
9. Fonctionnalités requises	12
9.1. Gestion des joueurs	12
9.2. Gestion des tournois	12
9.3. Consultation et affichage	12
9.4. Exemple de menu attendu	13

IV. Rapport et livrables	14
10. Structure du rapport	14
11. Livrables à rendre	14
12. Grille d'évaluation	14
Annexe — Rappels SQL utiles	16

1. Présentation du sujet

L'e-sport (sport électronique) est aujourd'hui une industrie mondiale générant des milliards de dollars de revenus. Des compétitions organisées autour de jeux comme *League of Legends*, *Valorant*, *CS2* ou *FIFA* réunissent des milliers de joueurs et des millions de spectateurs.

Dans ce projet, vous allez concevoir et implémenter une base de données pour gérer une **plateforme de tournois e-sport**, ainsi qu'une application Java permettant d'interagir avec celle-ci.

Le système devra permettre de gérer :

- des **jeux** et leurs caractéristiques,
- des **joueurs** et leurs équipes,
- des **tournois** et leur organisation en phases,
- des **matches** et leurs résultats,
- des **statistiques** de performance.

2. Description du domaine

2.1. Les jeux

Chaque jeu possède un nom, un genre (FPS, MOBA, Sport, Battle Royale, etc.), l'éditeur qui l'a développé, et l'année de sortie. Un jeu peut faire l'objet de plusieurs tournois.

2.2. Les joueurs

Un joueur est caractérisé par un pseudonyme (unique), un nom, un prénom, une date de naissance, une nationalité et un niveau (classement Elo ou rang). Un joueur peut appartenir à une équipe *par jeu* : il peut être dans une équipe pour *Valorant* et dans une autre pour *CS2*.

2.3. Les équipes

Une équipe possède un nom, un logo (chemin vers une image), une date de création et un pays d'origine. Une équipe est composée de plusieurs joueurs **pour un jeu donné** : c'est la notion de **roster**. Un roster a une date de début et une date de fin (si l'équipe se sépare).

2.4. Les tournois

Un tournoi est organisé autour d'un jeu précis. Il possède un nom, une date de début, une date de fin, un type (*en ligne* ou *LAN*), une dotation en euros et un statut (À venir, En cours, Terminé). Un tournoi est structuré en **phases** : phase de groupe, quarts de finale, demi-finales, finale.

2.5. Les matches

Un match oppose deux équipes dans le cadre d'une phase d'un tournoi. Il possède une date, un score pour chaque équipe (nombre de manches gagnées) et un résultat (équipe gagnante). Plusieurs maps (ou parties) peuvent composer un match.

2.6. Les statistiques

Pour chaque match, on enregistre des statistiques individuelles pour chaque joueur : nombre de kills, de deaths, d'assists et un score de performance. Ces statistiques permettent de calculer des classements.

3. Travail demandé

Le projet est décomposé en **quatre parties** que vous devez réaliser dans l'ordre. Chaque partie s'appuie sur la précédente.

Première partie

Modélisation

4. Modèle Conceptuel de Données (MCD)

Rappel — MCD

Le Modèle Conceptuel de Données représente les entités du domaine et les associations entre elles, sans se préoccuper de l'implémentation. On y indique les cardinalités de chaque association.

4.1. Travail à réaliser

À partir de la description du domaine (Section 2), vous devez :

1. Identifier toutes les **entités** du domaine avec leurs attributs.
2. Identifier les **associations** entre les entités.
3. Préciser les **cardinalités** (min, max) de chaque côté de chaque association.
4. Identifier les **attributs portés par les associations** si nécessaire.

4.2. Contraintes obligatoires

Votre MCD devra respecter les contraintes suivantes :

- Un joueur peut appartenir à plusieurs équipes **mais pas pour le même jeu simultanément**.
- Un tournoi est associé à **un seul jeu**.
- Un match appartient à **une seule phase** d'un tournoi.
- Les statistiques sont liées à **un joueur dans un match** précis.
- Une équipe peut participer à **plusieurs tournois**.

Attention

Vous devrez justifier vos choix de cardinalités dans votre rapport. Un MCD sans justification sera pénalisé.

4.3. Livrable attendu

Un schéma MCD réalisé avec un outil de modélisation (*draw.io*, *Looping*, *MySQL Workbench*, etc.) ou à la main, inclus dans le rapport au format PDF.

5. Modèle Logique de Données (MLD)

Rappel — MLD

Le Modèle Logique de Données est la traduction du MCD en un ensemble de tables relationnelles. On y fait apparaître les clés primaires (soulignées) et les clés étrangères (avec flèches ou notation #).

5.1. Travail à réaliser

À partir de votre MCD, appliquez les règles de passage MCD $\rightarrow$ MLD :

1. Toute entité devient une **table**.
2. Toute association de type **(1,n) – (0,n)** ou **(1,1) – (0,n)** se traduit par une clé étrangère.
3. Toute association de type **(0,n) – (0,n)** se traduit par une **table de liaison**.
4. Les **attributs d'association** deviennent des attributs de la table de liaison.

5.2. Livrable attendu

La notation textuelle complète du MLD sous la forme :

NomTable(cle_primaire, attribut1, attribut2, #cle_etrangere)

Exemple de notation attendu :

```
Jeu(id_jeu, nom, genre, editeur, annee_sortie)
Equipe(id_equipe, nom, pays, date_creation)
Joueur(id_joueur, pseudo, nom, prenom, date_naissance,
        nationalite, #id_equipe)
```

Deuxième partie

Base de données SQL

6. Script de création (DDL)

SGBD cible

Le SGBD utilisé dans ce projet est **MySQL 8.x**. Votre script doit être exécutable sans erreur sur une installation standard de MySQL.

6.1. Travail à réaliser

À partir de votre MLD, rédigez un script SQL complet qui :

1. Crée la base de données et la sélectionne :

```
CREATE DATABASE IF NOT EXISTS esport;
USE esport;
```

2. Supprime les tables si elles existent déjà (pour permettre la ré-exécution) :

```
DROP TABLE IF EXISTS Statistique;
DROP TABLE IF EXISTS Match_Esport;
-- etc.
```

3. Crée toutes les tables avec :

- les **clés primaires** appropriées,
- les **clés étrangères** avec les clauses `ON DELETE` et `ON UPDATE`,
- les **contraintes** `NOT NULL`, `UNIQUE`, `CHECK` pertinentes,
- le moteur `InnoDB` pour respecter les contraintes d'intégrité référentielle.

6.2. Exemple de structure attendue

```
CREATE TABLE Jeu (
  id_jeu      INT AUTO_INCREMENT,
  nom         VARCHAR(50) NOT NULL,
  genre       ENUM('FPS', 'MOBA', 'Sport', 'Battle Royale', 'Autre')
             NOT NULL,
  editeur     VARCHAR(50),
  annee_sortie YEAR,
  PRIMARY KEY (id_jeu),
  CONSTRAINT uk_jeu_nom UNIQUE (nom)
) ENGINE=InnoDB;
```

6.3. Jeu de données (DML)

Votre script devra également contenir des `INSERT INTO` pour peupler chaque table avec **au minimum** :

Table	Nombre minimum de lignes
Jeu	4
Equipe	6
Joueur	20
Tournoi	3
Phase	9 (3 par tournoi)
Match_Esport	15
Statistique	60

Important

Le jeu de données doit être **cohérent** : les scores des matchs doivent correspondre aux statistiques, les phases doivent s'enchaîner logiquement, etc. Un jeu de données incohérent sera pénalisé.

7. Requêtes SQL

Vous devez répondre aux **quinze requêtes** suivantes en SQL. Chaque requête doit être accompagnée d'un bref commentaire expliquant l'approche choisie.

7.1. Requêtes de base (SELECT, WHERE, ORDER BY)

- R1.** Afficher la liste de tous les joueurs (pseudo, nom, prénom, nationalité) triés par ordre alphabétique de pseudo.
- R2.** Afficher tous les tournois dont la dotation est supérieure à 10 000 €, en affichant le nom du tournoi, le jeu associé et la dotation.
- R3.** Afficher tous les matchs d'un tournoi donné (passé en paramètre), avec la date, les deux équipes et le score.

7.2. Requêtes avec jointures

- R4.** Afficher pour chaque équipe la liste de ses joueurs avec leur pseudo et leur rôle, pour un jeu donné.
- R5.** Afficher le nom du vainqueur de chaque tournoi terminé, avec le nom du tournoi et la dotation remportée.
- R6.** Pour chaque match, afficher les statistiques agrégées des deux équipes (total kills, total deaths par équipe).

7.3. Requêtes avec agrégats (GROUP BY, HAVING)

- R7.** Afficher le nombre de tournois organisés par jeu, trié par ordre décroissant.
- R8.** Afficher les équipes ayant participé à **au moins 2 tournois différents**.
- R9.** Afficher le joueur ayant le meilleur ratio kills/deaths moyen sur l'ensemble de ses matchs (pour les joueurs ayant joué au moins 5 matchs).
- R10.** Pour chaque tournoi, afficher la moyenne des scores de performance des joueurs par équipe.

7.4. Requêtes avancées (sous-requêtes, EXISTS, NOT EXISTS)

- R11.** Afficher les joueurs qui n'ont **jamais** participé à un tournoi de type LAN.

- R12.** Afficher les équipes qui ont gagné **tous leurs matchs** lors de la phase de groupe d'un tournoi donné.
- R13.** Afficher le classement des équipes dans un tournoi donné, basé sur le nombre de victoires, avec départage sur la différence de rounds gagnés.
- R14.** Afficher les joueurs qui ont participé à des tournois sur **au moins deux jeux différents**.
- R15.** Afficher, pour chaque jeu, le joueur ayant le plus grand nombre de kills cumulé sur tous ses matchs. En cas d'égalité, afficher les deux joueurs.

Conseil

Les requêtes R11 à R15 sont volontairement complexes et nécessitent l'utilisation de sous-requêtes corrélées, de `EXISTS` / `NOT EXISTS`, ou de fonctions de fenêtrage (`RANK`, `DENSE_RANK`). Elles sont valorisées dans la notation.

Troisième partie

Application Java

8. Architecture de l'application

Vous devez développer une application Java qui se connecte à votre base de données MySQL via **JDBC** et propose une interface permettant d'interagir avec les données.

8.1. Connexion JDBC

Rappel — JDBC

JDBC (*Java Database Connectivity*) est l'API standard Java pour interagir avec une base de données relationnelle. Vous aurez besoin d'ajouter le connecteur MySQL dans votre projet (`mysql-connector-j-x.x.x`).

La connexion doit être centralisée dans une classe `ConnexionBDD` :

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConnexionBDD {
    private static final String URL = "jdbc:mysql://localhost:3306/esport";
    private static final String USER = "root";
    private static final String PASS = "votre_mot_de_passe";

    private static Connection instance = null;

    public static Connection getConnection() throws SQLException {
        if (instance == null || instance.isClosed()) {
            instance = DriverManager.getConnection(URL, USER, PASS);
        }
        return instance;
    }

    public static void fermer() throws SQLException {
        if (instance != null && !instance.isClosed()) {
            instance.close();
        }
    }
}
```

8.2. Organisation du code

Vous devrez organiser votre code selon une architecture en couches :

Package	Rôle	Exemple
modele	Classes métier (POJO)	Joueur.java, Equipe.java
dao	Accès aux données (SQL)	JoueurDAO.java
vue	Interface utilisateur	MenuPrincipal.java
util	Utilitaires	ConnexionBDD.java

9. Fonctionnalités requises

L'application doit proposer un **menu textuel** (console) permettant d'accéder aux fonctionnalités suivantes.

9.1. Gestion des joueurs

```
// Exemple de méthode DAO attendue
public class JoueurDAO {
    // Ajouter un nouveau joueur
    public boolean ajouterJoueur(Joueur j) throws SQLException { ... }

    // Lister tous les joueurs
    public List<Joueur> listerJoueurs() throws SQLException { ... }

    // Rechercher un joueur par pseudo
    public Joueur rechercherParPseudo(String pseudo)
        throws SQLException { ... }

    // Modifier les informations d'un joueur
    public boolean modifierJoueur(Joueur j) throws SQLException { ... }

    // Supprimer un joueur
    public boolean supprimerJoueur(int idJoueur)
        throws SQLException { ... }
}
```

9.2. Gestion des tournois

L'application doit permettre :

- la création d'un nouveau tournoi (nom, jeu, dates, type, dotation),
- l'inscription d'une équipe à un tournoi,
- la saisie du résultat d'un match (score des deux équipes),
- la saisie des statistiques individuelles d'un match.

9.3. Consultation et affichage

L'application doit permettre :

- d'afficher le classement des équipes d'un tournoi,
- d'afficher le palmarès d'un joueur (tournois joués, victoires),
- d'afficher les statistiques globales d'une équipe,
- de rechercher un joueur ou une équipe par mot-clé.

9.4. Exemple de menu attendu

```
=====
Plateforme E-sport --- Menu Principal
=====
1. Gestion des joueurs
2. Gestion des équipes
3. Gestion des tournois
4. Saisir le résultat d'un match
5. Classement d'un tournoi
6. Statistiques d'un joueur
7. Quitter
=====
Votre choix : _
```

Conseil — Prepared Statements

Utilisez systématiquement des `PreparedStatement` plutôt que des `Statement` pour construire vos requêtes SQL, afin d'éviter les injections SQL et d'améliorer les performances.

```
String sql = "SELECT * FROM Joueur WHERE pseudo = ?";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, pseudo);
ResultSet rs = ps.executeQuery();
```

Quatrième partie

Rapport et livrables

10. Structure du rapport

Votre rapport (format PDF, 15–25 pages) doit comporter les sections suivantes :

1. **Page de garde** : noms, prénoms, groupe, année universitaire
2. **Introduction** : présentation du contexte et du sujet
3. **Analyse du domaine** : description des entités et de leurs relations
4. **MCD** : schéma + justification des cardinalités
5. **MLD** : notation textuelle complète
6. **Script SQL** : extraits commentés + justification des contraintes
7. **Requêtes SQL** : les 15 requêtes avec résultats et explications
8. **Application Java** : architecture, choix techniques, captures d'écran
9. **Conclusion** : bilan, difficultés rencontrées, améliorations possibles

11. Livrables à rendre

Vous déposerez sur Moodle une archive `.zip` nommée `NOM1_NOM2_Projet_BDD.zip` contenant :

Fichier / Dossier	Contenu	Format
<code>rapport.pdf</code>	Rapport complet	PDF
<code>script_creation.sql</code>	DDL + DML	SQL
<code>requetes.sql</code>	Les 15 requêtes	SQL
<code>src/</code>	Code source Java	Java
<code>README.txt</code>	Instructions de lancement	Texte

Attention

Tout projet dont le script SQL ne s'exécute pas sans erreur, ou dont l'application Java ne se lance pas, sera fortement pénalisé. Testez votre rendu sur une installation propre avant de le déposer.

12. Grille d'évaluation

Critère	Points	Détail
MCD	4	Entités, associations, cardinalités, justifications
MLD	2	Traduction correcte du MCD, notation
Script SQL (DDL)	4	Tables, contraintes, clés étrangères
Jeu de données (DML)	2	Cohérence, volume minimum
Requêtes SQL (R1–R10)	4	Exactitude, lisibilité
Requêtes SQL (R11–R15)	4	Complexité, sous-requêtes, agrégats
Application Java	6	Connexion JDBC, fonctionnalités, qualité du code
Rapport	2	Clarté, structure, orthographe
Soutenance	2	Maîtrise du sujet, réponses aux questions
Total	30	

Bonus

- **+1 point** : Interface graphique Java (Swing ou JavaFX) à la place du menu console.
- **+1 point** : Utilisation de vues SQL (`CREATE VIEW`) pour simplifier certaines requêtes complexes.
- **+1 point** : Mise en place de triggers SQL (ex : mise à jour automatique du classement après un match).

Annexe — Rappels SQL utiles

Fonctions d'agrégation

```
-- Comptage, somme, moyenne, min, max
SELECT COUNT(*), SUM(kills), AVG(kills), MIN(kills), MAX(kills)
FROM Statistique
WHERE id_match = 1;

-- Groupement avec filtre sur le groupe
SELECT id_joueur, AVG(kills) AS moy_kills
FROM Statistique
GROUP BY id_joueur
HAVING AVG(kills) > 10;
```

Sous-requêtes

```
-- Joueurs dont le ratio kills/deaths dépasse la moyenne globale
SELECT pseudo FROM Joueur
WHERE id_joueur IN (
    SELECT id_joueur FROM Statistique
    GROUP BY id_joueur
    HAVING AVG(kills) > (SELECT AVG(kills) FROM Statistique)
);
```

EXISTS / NOT EXISTS

```
-- Équipes n'ayant jamais perdu un match
SELECT nom FROM Equipe e
WHERE NOT EXISTS (
    SELECT 1 FROM Match_Esport m
    WHERE m.id_equipe_perdante = e.id_equipe
);
```

Fonctions de fenêtrage (MySQL 8+)

```
-- Classement des joueurs par kills dans chaque tournoi
SELECT
    j.pseudo,
    t.nom AS tournoi,
    SUM(s.kills) AS total_kills,
    RANK() OVER (PARTITION BY t.id_tournoi
                 ORDER BY SUM(s.kills) DESC) AS classement
FROM Statistique s
JOIN Joueur j      ON s.id_joueur = j.id_joueur
JOIN Match_Esport m ON s.id_match  = m.id_match
JOIN Phase p       ON m.id_phase   = p.id_phase
JOIN Tournoi t      ON p.id_tournoi = t.id_tournoi
GROUP BY j.id_joueur, t.id_tournoi;
```